

Network — ZTP, Underlay, and RoCEv2 Lossless Design

Rev 0.2 · 2026-07-19

Generated from Markdown — edit the .md source and regenerate (python3 docs/md2pdf.py)

Contents

1	Switch inventory and management plan	3
2	Day-0 dependency chain	4
3	ZTP architecture	5
3.1	Components	5
3.2	Zero-day: how the OOB switches come up (answer to the bootstrap question)	5
3.3	Spines and rail leaves — ZTP via OOB relay	5
3.4	Generated dnsmasq (relevant lines)	6
3.5	ztp.py behavior contract	6
3.6	Re-ZTP / recovery runbook	7
4	OOB network design (7010TX-48 pair)	8
5	Fabric underlay	9
5.1	IP plan (from 10.30.0.0/16 container, seed IPAM)	9
5.2	eBGP plan	9
5.3	MTU	9
6	RoCEv2 lossless design	10
6.1	Traffic classes	10
6.2	PFC	10
6.3	ECN (DCQCN marking)	10
6.4	Buffer / headroom math	10
6.5	ECMP and DLB	11
6.6	Host side (day-2, GPU/Network Operator)	11
6.7	Explicitly <i>not</i> lossless	11
7	Storage / services attachment (day-1)	12
8	Verification and acceptance	13
8.1	ZTP gates (per switch)	13
8.2	Fabric gates	13
8.3	RoCE gates (burn-in, with hosts)	13
9	Telemetry to the ops stack	14
10	What this doc does <i>not</i> cover	15
11	References	16
12	Revision History	17

Status: Draft

Scope: Day-0 switch bring-up (ZTP) for OOB / spine / rail-leaf switches, fabric underlay, RoCEv2 lossless Ethernet profile

Implements: SPEC.md §5 (network), §5.10 (OOB) · consumed by docs/build-guide.md §9 and docs/bootstrap.md

SoT rule: device mgmt IPs, MACs, and cables come from NetBox (bootstrap/netbox/seed/site.yaml pre-go-live). Switch configs are **rendered**, never hand-written: bootstrap/seed/ztp/render.py → one startup-config per switch serial.

1 Switch inventory and management plan

Device	Model	Rack	Ma1 IP (VLAN 20)	Ma1 MAC ¹	BGP ASN	Loopback0
oob-sw1	7010TX-48	GPU-1	10.20.0.11	00:00:00:00:23:01	— (MLAG/L2)	—
oob-sw2	7010TX-48	GPU-2	10.20.0.12	00:00:00:00:23:02	— (MLAG/L2)	—
spine1	7060DX5-64S	BOOT	10.20.0.13	00:00:00:00:21:01	65000	10.30.0.1/32
spine2	7060DX5-64S	BOOT	10.20.0.14	00:00:00:00:21:02	65000	10.30.0.2/32
leaf-rail0-7	7060DX5-32	GPU-1/2	10.20.0.41-48	00:00:00:00:22:0 <i>r</i>	65101-65108	10.30.0.11-18/32

¹ Placeholder MACs from the seed — replaced with burned-in MACs at receiving (build-guide §5); DHCP reservations and `render.py` output regenerate from NetBox.

Gateways (VARP on the OOB MLAG pair): **10.10.0.1** (VLAN 10 mgmt), **10.20.0.1** (VLAN 20 OOB). All infra services (DHCP/DNS/NTP/HTTP) live on **seed01 = 10.10.0.10**.

2 Day-0 dependency chain

```
seed01 (hand-installed)           ← the only manual install
| DHCP option 67 → http://10.10.0.10/ztp/ztp.py
├─ oob-sw1, oob-sw2   (zero-day bench ZTP, §3.2)   ← first switches
├─ spine1, spine2     (ZTP via 00B relay, §3.3)
└─ leaf-rail0...7     (ZTP via 00B relay, §3.3)
```

The chicken-and-egg rule: every switch except the seed is ZTP'd; nothing is configured by hand except seed01 and emergency console access.

3 ZTP architecture

3.1 Components

Piece	Where	Role
dnsmasq	seed01 (/etc/dnsmasq.d/ ai-cluster*.conf)	DHCP + DNS for VLAN 10/20; option 67 for switches
nginx	seed01 (/var/www/html/ztp/)	Serves ztp.py, configs/ <serial>, images/
ztp.py	bootstrap/seed/ztp/ztp.py → http://10.10.0.10/ztp/ztp.py	On-switch bootstrap: serial → config → reload
render.py	bootstrap/seed/ztp/render.py	Renders 12 startup-configs from seed/site.yaml
serialmap.yaml	bootstrap/seed/ztp/serialmap.yaml	Serial → hostname (filled at staging)

render.py output is derived from the same seed that drives NetBox, cabling, and dnsmasq — OOB access-port maps come from all_cables(), so **the config cannot disagree with the cabling guide**.

Standard EOS ZTP mechanics used: a switch with no startup-config broadcasts DHCPDISCOVER on Management1 **and** on any front-panel port with link; option 67 (bootfile) containing an http:// URL is downloaded and executed as a script; the script writes startup-config, sets /mnt/flash/zerotouch-config DISABLE=True, and reloads.

3.2 Zero-day: how the OOB switches come up (answer to the bootstrap question)

The OOB pair is the root of the management network, so it cannot depend on any other switch. Procedure (also in build-guide §9.1):

1. **seed01 first.** Installed by hand (autoinstall USB / BMC virtual media) on the bench; 00-seed-host.sh brings up dnsmasq + nginx. seed01 needs exactly one Ethernet port.
2. **Fill serialmap.yaml** with the two 7010TX serials (captured at receiving) and run render.py → /var/www/html/ztp/configs/<serial>.
3. **Bench ZTP oob-sw1:** patch seed01 eth0 → oob-sw1 any front-panel port (e.g. Ethernet48), power on. With no startup-config, EOS ZTP DHCPs untagged on the front port; dnsmasq answers from the dynamic pool (10.10.0.200–250) and returns option 67 via the **catch-all dhcp-boot** line. ztp.py runs, learns its serial, pulls configs/<serial> (the real oob-sw1 config: MLAG, VLANs, SVIs, VARP, DHCP relay), writes it, disables ZTP, reloads.
4. **Repeat for oob-sw2** (direct to seed01, or via an oob-sw1 access port once sw1 is live).
5. **Rack both**, cable per docs/cabling.md §5 (access ports + MLAG peer-link Ethernet49–50), power, verify MLAG: show mlag → state: active, peer: live; VARP gateways .1 answer on both VLANs.
6. **Fallback:** if ZTP fails twice, console in (9600 8N1, console server in BOOT) and paste the rendered config for that serial — it is identical to what ZTP would have delivered.

Why bench-ZTP instead of manual console config: the OOB pair's config is the most intricate in the design (MLAG + VARP + relay + port map); delivering it through the same rendered pipeline as everything else removes a whole class of day-0 typos. The bench step needs no infrastructure beyond seed01 and one patch cable.

3.3 Spines and rail leaves — ZTP via OOB relay

Once the OOB pair is live:

```

spine/leaf Management1 —cat6→ 00B access port (VLAN 20)
| DHCPDISCOVER
  ▼ relay: ip helper-address 10.10.0.10 (Vlan20 SVI, VARP 10.20.0.1)
dnsmasq (reservation by Ma1 MAC → 10.20.0.13/14, .41–48, tag ztp)
| option 67 → http://10.10.0.10/ztp/ztp.py
  ▼
ztp.py → configs/<serial> (underlay + RoCE profile) → reload

```

- Reservations are keyed by **Ma1 MAC** (seed placeholders until staging; regenerated by `netbox-sync.sh` after MAC capture).
- Config identity is by **serial** (not IP), so a swap/replacement unit just works once its serial is in `serialmap.yaml` (or NetBox post-go-live).
- **Order matters:** spines before leaves is conventional but not required — leaves simply retry until spines appear. Bring up spine1, spine2, then all 8 leaves; BGP comes up as pairs complete.

3.4 Generated dnsmasq (relevant lines)

```

dhcp-range=10.10.0.200,10.10.0.250,12h          # bench/ZTP dynamic pool (VLAN 10)
dhcp-range=10.20.0.0,static,12h                 # VLAN 20: reservations only (via relay)

dhcp-boot=tag:!ipxe,tag:efi64,ipxe.efi          # servers: UEFI → iPX E → OS
dhcp-boot=tag:ipxe,http://10.10.0.10/boot.ipxe
dhcp-boot=tag:ztp,http://10.10.0.10/ztp/ztp.py   # switches w/ Ma1 reservations
dhcp-boot=tag:!ztp,tag:!efi64,tag:!ipxe,http://10.10.0.10/ztp/ztp.py # bench catch-all

# from generated/dhcp-hosts.conf (netbox-sync.sh):
dhcp-host=00:00:00:00:21:01,set:ztp,10.20.0.13,spine1,infinite
dhcp-host=00:00:00:00:22:03,set:ztp,10.20.0.44,leaf-rail3,infinite
dhcp-host=00:00:00:00:01:02,10.20.0.101,worker01-bmc,infinite
...

```

3.5 ztp.py behavior contract

Step	Action	Failure handling
1	show version → serial	exit non-zero, stays in ZTP, retries next boot
2	GET configs/<serial> (5× backoff)	as above — fix serialmap and power-cycle
3	Optional !IMAGE: marker → stage + sha512-verify EOS image, write boot- config	abort on checksum mismatch
4	Write /mnt/flash/ startup-config	—
5	zerotouch-config DISABLE=True, reload	lands on real config; ZTP never re-fires

Log: `/mnt/flash/ztp.log` on the switch; seed-side hits in nginx access log.

3.6 Re-ZTP / recovery runbook

Goal	Steps
Re-provision a switch	write <code>erase</code> → <code>reload</code> → ZTP runs again (config still in <code>configs/<serial></code>)
Skip ZTP once	console: <code>zerotouch cancel</code>
New/replacement unit	add serial to <code>serialmap.yaml</code> (or NetBox), <code>render.py</code> , <code>netbox-sync.sh</code> ; power on
Change config fleet-wide	edit seed/templates → <code>render.py</code> → <code>config replace</code> via eAPI/SSH per switch (day-2: CloudVision or Ansible)
seed01 down during ZTP storm	switches retry (backoff in <code>ztp.py</code>); nothing bricks — ZTP re-fires on next boot

4 OOB network design (7010TX-48 pair)

Two 1G switches as an **MLAG pair** across GPU-1/GPU-2 — a single OOB switch failure must not black-hole BMC access (SPEC §5.10). **No lossless QoS on OOB** — it carries best-effort management only.

Element	Design
Peer-link	Ethernet49–50 = Port-Channel1 (2× 25G SFP28 DAC), trunk group mlagpeer
MLAG peer	Vlan4094 10.255.0.0/30 (.1 sw1, .2 sw2), domain-id oob
VLAN 10 (mgmt)	server mgmt0 (OS/PXE); SVI .11/.12, VARP 10.10.0.1
VLAN 20 (oob)	BMC + switch Ma1 (ZTP); SVI .11/.12, VARP 10.20.0.1 , ip helper-address 10.10.0.10
Uplinks	SFP28 51–52 reserved for site network/border (day-1: none)
Control plane	ACL mgmt-plane: only 10.10/10.20 + ICMP to switch CPU
STP	MSTP, sw1 root (4096) / sw2 backup (8192); none on VLAN 4094

Access-port map (rendered from cabling SoT — docs/cabling.md §5):

oob-sw1 ports	Use	VLAN
Ethernet1–10	BMC: cp01–03, util01–03, worker01–04	20
Ethernet11–21	mgmt0: seed01, cp01–03, util01–03, worker01–04	10
Ethernet22–27	Ma1: spine1, spine2, leaf-rail0–3	20
oob-sw2 ports	Use	VLAN
Ethernet1–4	BMC: worker05–08	20
Ethernet5–8	mgmt0: worker05–08	10
Ethernet9–12	Ma1: leaf-rail4–7	20

Headroom: 27/48 and 12/48 ports used — STOR rack and site uplinks fit without a third OOB switch (SPEC §5.10 “optional 3rd” not needed day-1).

Full rendered configs: `render.py` output for `SERIAL-00B-SW1/2` (MLAG, VARP, relay, ACL, complete port map).

5 Fabric underlay

Day-1: **L3 leaf-spine, eBGP numbered** underlay. The EVPN/VXLAN overlay (vrf **storage** + vrf **edge**, VTEPs, border) is layered on top — design in [docs/overlay.md](#); RoCE rail traffic stays native in the underlay (never VXLAN-encapped). Spine ports 34–64 remain reserved.

5.1 IP plan (from 10.30.0.0/16 container, seed IPAM)

Block	Use
10.30.0.0/24	Loopbacks: spines .1/.2, leaf-rail0–7 .11–.18
10.30.0.0/19	p2p /31 pool: leaf-rail i uses 10.30.{2i+1}.0/29 → spine1 (4× /31) and 10.30.{2i+2}.0/29 → spine2
10.30.64.0/19	Host rail subnets: rail i = 10.30.{64+i}.0/24 (leaf .1, worker0N .1N)
10.40.0.0/16	Storage (routed ports on leaf-rail0/rail7, §7)

Numbered /31s (not unnumbered) chosen deliberately: they are NetBox-IPAM-native, ping/LLDP-auditable per link, and unambiguous in ZTP-rendered configs — at 64 links the address economy of unnumbered buys nothing.

Example — leaf-rail3: uplinks Ethernet17–24 = 10.30.7.0/31 (spine1), 10.30.8.0/31 (spine2), 10.30.7.2/31, 10.30.8.2/31, 10.30.7.4/31, 10.30.8.4/31, 10.30.7.6/31, 10.30.8.6/31; rail VLAN 67 SVI 10.30.67.1/24.

5.2 eBGP plan

Knob	Value
ASN	spines 65000; leaf-rail i = 65101+ i
Sessions	32 per spine (8 leaves × 4), 8 per leaf (4 per spine)
ECMP	maximum-paths 32
BFD	on every p2p neighbor
Advertised	leaf: loopback /32 + rail /24; spine: loopback /32 (no transit filtering needed at this scale)
TTL/next-hop	eBGP single-hop; no communities day-1

Hosts are *not* BGP speakers day-1: rail subnets are leaf SVIs (L2 access ports); worker NIC IPs (10.30.{64+rail}.1N) are assigned at OS provisioning from NetBox data (Metal3/cloud-init — bootstrap track).

5.3 MTU

9216 on every fabric port (leaf host + uplink, spine) and rail SVI; host NICs 9214/9000 payload tuned with the GPU stack. Verified end-to-end in burn-in (ping -M do -s 9172 per rail).

6 RoCEv2 lossless design

MoE expert-parallel all-to-all is bursty and many-to-one (incast) per rail; the design target is **zero loss for the RoCE class** with early ECN so DCQCN rarely lets PFC fire.

6.1 Traffic classes

Class	DSCP	Traffic-class	Queue behavior	Used by
RoCE data	26	3	PFC no-drop + ECN (WRED mark)	NCCL/EP (GPU rail NICs)
CNP	48	6	Strict priority (small)	DCQCN congestion notifications
Storage	18	1	Weighted, lossy	model FS / archive (§7)
Default	0	0	Best-effort	everything else
Network control	—	(EOS default)	protected	BGP/BFD/LLDP

Marking source of truth is the **host** (NIC marks DSCP 26 on RoCE, 48 on CNP); switches run `qos trust dscp` on all fabric ports and map per the table. VLAN CoS is irrelevant here (routed uplinks; rail access ports carry one class).

6.2 PFC

- `priority-flow-control priority 3 no-drop` on **every** fabric port (leaf host + uplink, spine downlink) — lossless must be symmetric or head-of-line blocking just moves.
- **PFC watchdog** enabled globally (`polling-interval 0.4`, `action errdisable`): a stuck pause (misbehaving NIC/host) errdisables the port instead of tree-pausing the rail. Alerts to observability stack; port auto-recovers after host fix.
- Pause *received* on TC0/1/6 classes is invalid — only TC3 participates.

6.3 ECN (DCQCN marking)

WRED-ECN on TC3, all fabric ports — starting values, tuned with LANZ during burn-in (§8):

Knob	Start	Reasoning
min-threshold	512 KB	mark early: $400\text{G} \times \sim 3\ \mu\text{s}$ fabric RTT $\approx$ 150 KB BDP; well below PFC headroom
max-threshold	1536 KB	$\approx 3 \times$ min; queue beyond this $\Rightarrow$ mark $\sim$ all $\Rightarrow$ DCQCN clamps rate before PFC
drop-probability	100 (mark-only above max)	RoCE class never tail-drops

Goal: **PFC frames $\approx$ 0 in steady state**; ECN marks carry the congestion signal. If PFC fires routinely, thresholds or host DCQCN gains are wrong — not “normal”.

6.4 Buffer / headroom math

Per-port PFC headroom $\approx$ link rate $\times$ (PFC reaction time) + $2 \times$ MTU:

$400 \text{ Gbps} \times 5 \mu\text{s} \text{ (MAC + ~3 m DAC + ASIC pipeline)} \approx 250 \text{ KB} + 18 \text{ KB} \approx \sim 300 \text{ KB per port}$
 leaf: $16 \text{ fabric ports} \times 300 \text{ KB} \approx 5 \text{ MB} \ll 57 \text{ MB shared (DX5-32)}$
 spine: $32 \text{ ports} \times 300 \text{ KB} \approx 10 \text{ MB} \ll 114 \text{ MB shared (DX5-64S)}$

MMU/headroom-pool commands are platform/release-specific (Tomahawk-class on DX5) — **pin exact knobs during lab validation** against the target EOS release; the rendered configs carry the queue/ECN/PFC intent plus this budget. Worst-case check at burn-in: all 8 hosts blasting one rail egress must not drop in TC3.

6.5 ECMP and DLB

- Static-hash ECMP entropy: `port-channel load-balance` fields include L3 src/dst + L4 ports (RoCE QP per-connection gives flow entropy).
- **DLB (Dynamic Load Balancing)** on DX5 platforms where supported by the target EOS release: flowlet-based rebalancing for long-lived AI flows over the 8-way leaf→spine mesh; fall back to resilient ECMP if the release lacks it. Exact `load-balance` policies syntax pinned in lab validation (§8).

6.6 Host side (day-2, GPU/Network Operator)

Setting	Value
RoCE mode	v2, GID index per mlx (RoCEv2/IPv4)
DCQCN	enabled on all 8 rail NICs (ConnectX-7/BF3)
NIC QoS	trust DSCP egress; PFC on priority 3 only; RoCE DSCP 26, CNP DSCP 48
NCCL	<code>NCCL_IB_GID_INDEX</code> (RoCEv2), <code>NCCL_IB_TC=104</code> (DSCP 26 $\ll$ 2), rail HCA pinning per worker

Applied via NVIDIA Network Operator / `mlnx_qos` + `mlxconfig` on workers — outside this doc’s switch scope but listed because the lossless contract is end-to-end: a host that marks nothing DSCP 26 gets no lossless treatment.

6.7 Explicitly *not* lossless

- **OOB (7010TX-48)**: no PFC/ECN anywhere (SPEC §5.10).
- **Storage class (TC1)**: lossy by design; storage retries, GPU collectives must not wait behind storage bursts.
- **Storage VLAN/VRF segmentation** stays separate from the RoCE class queues (§7).

7 Storage / services attachment (day-1)

Item	Design
Ports	leaf-rail0 Ethernet25–28, leaf-rail7 Ethernet25–28 (routed, MTU 9216)
Addressing	/31 p2p per link from 10.40.0.0/24; storage nodes 2× 400G each to both leaves (ECMP)
QoS	DSCP 18 → TC1 (§6.1); no PFC; separate from RoCE class
Routes	10.40.0.0/16 lives in vrf storage (EVPN type-5) — no longer in the default VRF
Isolation	dedicated storage VRF via EVPN/VXLAN — done , docs/overlay.md §3

Model-load bandwidth: $4 \times 400\text{G} \approx 160\text{ GB/s}$ fabric-facing — above the SPEC §6.4 $\geq 40\text{ GB/s}$ hot-tier target with headroom.

8 Verification and acceptance

8.1 ZTP gates (per switch)

Gate	Command / check	Pass
ZTP done	<code>show zerotouch</code>	disabled, config present
Identity	<code>show hostname, serial ↔ serialmap</code>	exact
Image	<code>show version</code>	target EOS release
Ma1	<code>show interfaces Management1</code>	reserved 10.20.0.x, ping seed01
Time/DNS	<code>show ntp status, DNS resolve</code>	synced to 10.10.0.10

8.2 Fabric gates

Gate	Command	Pass
Links	<code>show interfaces status</code>	leaf: 8 host + 8 uplink connected; spine: 32 connected
BGP	<code>show bgp summary</code>	all 32/8 sessions Established
ECMP	<code>show ip route 10.30.0.11/32 (from another leaf)</code>	8-way
MTU	<code>ping -M do -s 9172 across each /31</code>	64/64
BFD	<code>show bfd peers</code>	all up

8.3 RoCE gates (burn-in, with hosts)

Gate	Method	Pass
Line rate per rail	<code>ib_write_bw</code> worker-pair per rail	~400G, retransmits ≈ 0
Incast	7→1 same-rail perfest	no TC3 drops; PFC ≈ 0 , ECN marks > 0
All-to-all	<code>nccl-tests all_to_all</code> 8×8	expected bus BW; no stalls
Watchdog drill	pause-flood one host port	errdisable fires, rail survives
Soak	2 h all-rail load	0 watchdog events, bounded ECN, 0 CRC

Counters watched: `show interfaces counters | include pause, show qos interfaces Ethernet*, LANS queue-depth streams, show interfaces counters errors`.

9 Telemetry to the ops stack

Signal	Source	Use
PFC pause tx/rx per port	EOS counters → Prometheus (eAPI/ON-CHANGE)	lossless pressure, storm early-warning
ECN marks per queue	EOS QoS counters	DCQCN health (marks good, pauses bad)
Queue depth	LANZ streaming	microburst/incast forensics
BGP/BFD state	EOS → alerts	fabric reachability
ZTP events	nginx/dnsmasq logs on seed01 + /mnt/flash/ztp.log	bring-up audit

10 What this doc does *not* cover

- Host/NIC DCQCN and NCCL tuning beyond §6.6 (GPU platform track)
 - EVPN/VXLAN overlay, VRFs, border/API ingress — → [docs/overlay.md](#)
 - Site/campus uplink for the OOB pair (SFP28 51–52 reserved)
 - Storage-product-specific networking (→ [docs/storage.md](#), planned)
-

11 References

Doc / path	Use
SPEC.md §5, §5.10	Fabric + OOB requirements
docs/overlay.md	EVPN/VXLAN overlay on this underlay
docs/cabling.md	169-cable tables incl. OOB/Ma1/peer
docs/build-guide.md §9	Physical switch bring-up sequence
bootstrap/seed/ztp/	ztp.py, render.py, serialmap.example.yaml
bootstrap/seed/dnsmasq.conf.tmpl + bootstrap/seed/generated/dhcp-hosts.conf	DHCP/option-67 plan as deployed
Arista EOS <i>ZTP</i> and <i>RoCE/AI networking</i> chapters (target release)	Syntax validation gate before production

12 Revision History

Version	Date	Notes
0.1	2026-07-19	Initial: ZTP day-0 (OOB bench ZTP → relay ZTP), numbered eBGP underlay, RoCEv2 lossless profile
0.2	2026-07-19	Overlay delivered (docs/overlay.md): storage moved to vrf storage; host ports trunked (native rail VLAN + tagged 50/200); border on spine Eth33